

Playback-Aware KV-Cache Management for Consistent Barge-In Handling in Cascaded Spoken Dialogue Systems

1st Given Name Surname
dept. name of organization
name of organization
City, Country
email address

2nd Given Name Surname
dept. name of organization
name of organization
City, Country
email address

Abstract—Cascaded pipelines (streaming ASR $\rightarrow$ LLM $\rightarrow$ streaming TTS) remain the mainstream architecture for spoken dialogue systems, yet they suffer from a structural consistency flaw under user barge-in: the LLM’s generation progress, the TTS synthesis progress, and the actual audio playback progress advance at different paces. If the system commits everything it has generated to the dialogue history, the model will later refer to content the user never heard. We formalize the generation–synthesis–playback progress pointers and the heard-boundary semantics, and build three mechanisms on top of them: (1) speculative generation scheduling that softens endpoint detection into an abortable process driven by continuous turn-detection confidence; (2) playback-aware KV-cache management that reverse-maps the playback position to a token boundary and performs explicit KV truncation with synchronized attention-mask and position-encoding correction plus chat-template role rebuilding; and (3) three policies for the interrupted history. On a 7B backbone with MultiWOZ-derived data, naive generation-point truncation lets unheard content resurface in up to 51% of interrupted turns (surface-overlap criterion; 2.7% under a conservative LLM-judge criterion), whereas our playback-aware truncation reduces the rate to zero by construction; the barge-in critical path is constantly sub-millisecond and independent of context length, with KV reuse up to $39.7\times$ faster than re-prefilling at 8k context; speculative scheduling yields a continuously tunable waste–latency frontier, cutting post-utterance first-token latency from 48.5 ms to 12.1 ms for about 4.5% token waste. To our knowledge this is the first open-source, reproducible cascaded implementation of playback-aware context management at the KV-cache level.

Index Terms—spoken dialogue system, barge-in, KV cache, speculative generation, context consistency, low latency

I. INTRODUCTION

Large language models (LLMs) have moved voice assistants from command-style interaction toward natural conversation. In engineering practice, the cascaded architecture—streaming automatic speech recognition (ASR), an LLM, and streaming text-to-speech (TTS)—remains the mainstream design thanks to its modular controllability and strong reasoning ability. Beyond first-token latency, however, natural conversation requires graceful handling of *barge-in*: users interrupt at any time, and whether the system can continue coherently depends on a

subtle question—*does what the system believes it said match what the user actually heard?*

In a cascade this mismatch is structural. The LLM generates tokens faster than the TTS synthesizes them, and synthesis in turn runs ahead of real-time playback. When the user interrupts mid-playback, a system that commits “everything generated” to the dialogue history will later refer to information the user never heard (“as I mentioned earlier...”), derailing the conversation. Our experiments show that under such naive truncation, unheard-content references occur in up to 51% of interrupted turns by a surface-overlap criterion (Sec. V).

The principle that *dialogue history should equal what the user actually heard* is already practiced by commercial systems—OpenAI’s Realtime API [1], Microsoft’s Azure Voice Live [2], and LiveKit Agents [3]—and we explicitly do *not* claim originality for the principle itself. These implementations, however, are closed-source or operate on managed transcript text; to our knowledge, no academic or open-source work realizes playback-aware context management *inside* LLM inference via explicit KV-cache operations. This paper fills that gap. Our contributions are:

- **Speculative generation scheduling.** Continuous turn-detection confidence with a speculation threshold turns endpoint detection into an abortable, roll-backable process; a nine-point waste–latency frontier on real data shows a knee where about 4.5% token waste cuts post-utterance first-token latency from 48.5 ms to 12.1 ms (Sec. IV-A, V).
- **Playback-aware KV-cache management (core).** A four-way mapping timeline (token $\leftrightarrow$ segment $\leftrightarrow$ audio chunk $\leftrightarrow$ sample span), explicit truncation via `DynamicCache.crop` with synchronized mask/position handling, chat-template role-boundary rebuilding, and speculation rollback. The barge-in critical path is measured sub-millisecond, independent of context length; KV reuse is up to $39.7\times$ faster than re-prefilling at 8k context (Sec. IV-B, V).
- **Interrupted-history policies.** Implementation and ablation of naive truncation, interruption marking, and

lightweight-model rewriting (Sec. IV-C, V).

- **An open-source, reproducible reference implementation** with a dual-criterion consistency protocol including human arbitration.

The high-level idea of contribution 2 is prior art in commercial systems; our novelty lies in the open, reproducible, mechanism-level realization and its quantitative evaluation. The soft trigger and the rewriter use off-the-shelf models; we claim no model-level contribution.

II. RELATED WORK

1) *Commercial practice*: OpenAI’s Realtime API provides `conversation.item.truncate`: on barge-in the client reports `audio_end_ms` and the server deletes unplayed audio and its transcript, explicitly “so the context does not contain text the user has not heard” [1]. Azure Voice Live offers `auto_truncate`, updating the conversation context to the played portion [2]. LiveKit Agents commits only the actually-forwarded transcript and marks the message interrupted [3]. Playback-aware truncation is thus established engineering practice; the differences to our work are open vs. closed source, explicit KV operations vs. transcript deletion, and measured playback vs. estimation—not whether the capability exists.

2) *Academic streaming dialogue*: LTS-VoiceAgent [4] reduces think-while-listening latency with semantic triggering and incremental reasoning, but does not address barge-in, playback position, or KV truncation. RelayS2S [5] speculatively drafts a response prefix with a fast duplex model and verifies the handoff to a slower cascade; it shares our “speculation for latency” spirit but targets onset latency, not interrupt invalidation or KV management. Moshi [6] models both speakers as parallel streams in a frame-synchronous end-to-end architecture, so “generation equals playback” holds implicitly—which, from the opposite direction, confirms that the three-pointer mismatch is specific to cascades. FireRedChat [7] improves barge-in *detection* with personalized streaming VAD but does not manage the half-spoken reply in the LLM context.

3) *KV-cache operations*: The HuggingFace `DynamicCache` exposes a `crop` primitive [8]; serving frameworks optimize prefix caching for throughput. Orthogonal agent-side work prunes or discards KV state driven by text or tool logic—IntentKV [9], speculative tool calling [10]—not by audio playback progress. We are not aware of a published open-source precedent that crops KV by playback position and rebuilds chat-template role boundaries in a speech cascade.

III. PROBLEM FORMULATION

A. Pipeline model and progress pointers

Consider a cascade $\mathcal{S} = \langle \text{ASR}, \text{LLM}, \text{CHK}, \text{TTS}, \text{PLY} \rangle$. Streaming ASR emits *final* (non-revisable) text segments $u_1, u_2, \dots$, prefilling the LLM’s KV cache $\mathcal{K}$ incrementally. The reply is generated autoregressively as tokens $y_1, \dots, y_G$; a sentence chunker CHK partitions the token stream into segments f_j with contiguous token spans

$[\text{ts}(f_j), \text{te}(f_j))$, the atomic unit of truncation. Streaming TTS renders each segment into audio chunks occupying sample span $[\text{ss}(f_j), \text{se}(f_j))$; the player reports the played-sample count p .

At any time t three pointers measure progress on the same reply: generation $g(t)$ (tokens), synthesis $s(t)$ (segments), playback $p(t)$ (samples). Since generation outpaces synthesis and synthesis outpaces real-time playback,

$$p(t) \leq s(t) \leq g(t) \quad \forall t. \quad (1)$$

Fig. 1 illustrates the mismatch. Frame-synchronous end-to-end models such as Moshi collapse the three pointers into one; a cascade must manage the gap explicitly.

B. Heard boundary and consistency

Let barge-in occur with p inside segment f_k ($\text{ss}(f_k) < p \leq \text{se}(f_k)$). The *heard boundary* is

$$H(p) = \text{te}(f_k), \quad (2)$$

i.e., the interrupted segment is rounded up into “heard.” The history is *user-perception consistent* iff the interrupted turn’s assistant content is kept exactly on $[0, H(p))$. The naive alternative keeps all G tokens, of which $y_{H(p)+1..G}$ are unheard. The *quantization error* $\varepsilon(p) = H(p) - p^*$ (with p^* the proportional token position of p inside f_k) is bounded by one segment; we do not hide this error but quantify it with a *strict* criterion below. Two truncation events share one mechanism: playback-time barge-in ($p > 0$) and speculation invalidation ($p = 0$, roll back to $H = 0$).

C. Reverse mapping and truncation legality

The reverse map $\Phi : p \rightarrow f_k \rightarrow [\text{ts}(f_k), \text{te}(f_k)) \rightarrow H(p)$ must resolve in near-constant time—it sits on the “stop-on-interrupt” critical path. Truncating $\mathcal{K}$ to absolute position $N = a_0 + H(p)$ (with a_0 the assistant-turn start) is *legal* iff (i) KV tensors, attention mask, and recorded sequence length agree at N ; (ii) subsequent position encodings continue from N ; (iii) chat-template role boundaries are rebuilt so later turns remain structurally valid. Conditions (ii) and (iii) are the main engineering pitfalls; all three are runtime-checked throughout our experiments.

D. Metrics

Latency: TFTT_{eff} measures true-end-of-speech to first usable token (user-perceived; surviving speculation makes it ≈ 0); mouth-to-ear is modeled as first-segment-ready plus the TTS first-chunk latency from a measured device profile. *Consistency*: the unheard-content reference rate under a *loose* criterion (complete unheard segments; zero under our method *by construction*—the experiment quantifies the baseline’s failure rate, not ours) and a *strict* criterion (adds the unplayed tail of the partial segment, i.e., the consequence of ε). Reference detection uses two instruments whose agreement we report: a lexical rule detector (systematically high; upper bound) and an LLM judge for specific-information reference (conservative;

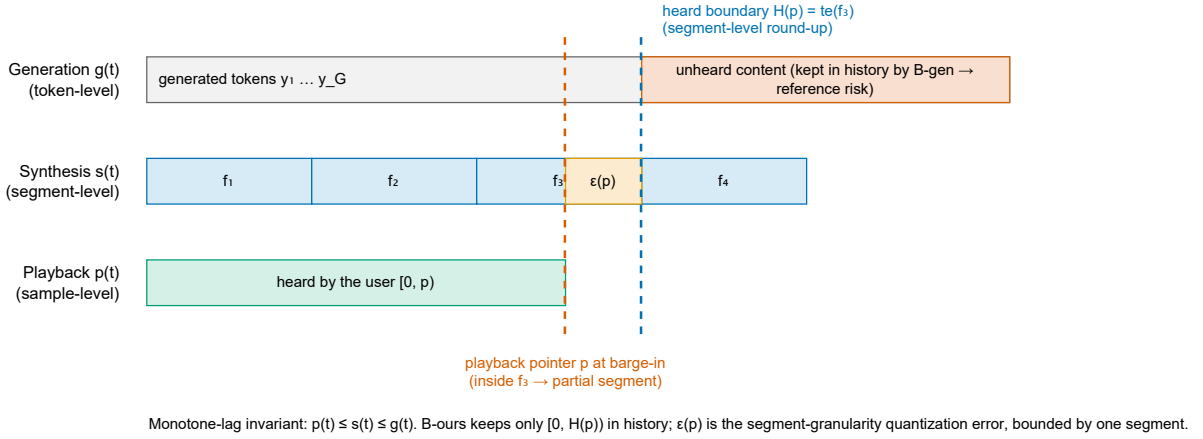


Fig. 1. Three progress pointers and the heard boundary. At barge-in the playback pointer p falls inside segment f_3 ; the heard boundary $H(p) = te(f_3)$ rounds up at segment granularity, and $\epsilon(p)$ is the bounded quantization error. Content between p and g is unheard; keeping it in history (B-gen) creates reference risk.

lower bound), plus human arbitration on a stratified sample.
Efficiency: the speculation waste rate

$$\rho = \frac{\sum \text{invalidated speculative tokens}}{\sum \text{invalidated speculative tokens} + \sum \text{final tokens}}, \quad (3)$$

and the $(\rho, TTFT_{\text{eff}})$ frontier under threshold sweep.

IV. SYSTEM DESIGN

Fig. 2 shows the overall architecture: the phase-one streaming-ASR + incremental-prefill pipeline is extended with an output-side three-stage pipeline (chunking $\rightarrow$ streaming TTS $\rightarrow$ playback) and two control mechanisms. Design principles: (P1) *history* = *what the user heard* (the principle itself is prior art; we contribute its explicit KV-level realization on an open cascade); (P2) *trade redundant, measurable computation for interaction fluency*.

A. Speculative Generation Scheduling

An off-the-shelf turn-detection model (TEN [11]) yields a continuous confidence c_i for the accumulated user text. When $c_i \geq \theta$ the system records the speculation start $b := |\mathcal{K}|$, injects the assistant role marker, and pre-generates up to B tokens (the speculation budget bounds single-shot invalidation cost). If another user segment arrives, the speculation is invalidated: KV is cropped back to b (a degenerate case of playback truncation with $p = 0$) and the output counts toward ρ . If the user has truly finished and a speculation survives, cached tokens are emitted immediately ($TTFT_{\text{eff}} \approx 0$) and generation continues. The trigger runs on the text side in parallel with the segment’s KV prefill, adding no end-to-end latency. Sweeping θ turns the endpoint decision into a tunable knob along the $(\rho, TTFT_{\text{eff}})$ frontier.

B. Playback-Aware KV-Cache Management

1) *Four-way mapping timeline*: An append-only structure keyed by segment records $\langle f_j, [ts, te], \text{chunk list}, [ss, se], \text{state} \rangle$ with states speculative $\rightarrow$

synthesizing $\rightarrow$ enqueued $\rightarrow$ playing $\rightarrow$ played/invalidated. Three producers write their own dimensions (generation thread: token $\leftrightarrow$ segment; TTS thread: segment $\leftrightarrow$ chunks and the sample axis; player thread: atomic update of p). Since segments per turn are few, one table lock plus a lock-free playback cursor meets the sub-millisecond lookup requirement. Token spans come from whitespace-insensitive character-count alignment between the chunker’s text and the decoded token stream, verified lossless.

2) *Legal truncation*: On barge-in, $\Phi(p)$ yields $H(p)$ and the cache is cropped to N : KV tensor slicing (`DynamicCache.crop` [8]), *synchronized* attention-mask shortening to N (the mask length is the source of truth for subsequent past length—cropping KV without the mask silently corrupts position encoding), and ledger updates so later prefills recompute positions explicitly from N . The critical path is exactly this tensor slicing: sub-millisecond.

3) *Role-boundary rebuild*: KV is a flat sequence; role structure exists only in chat-template markers. We derive the role-switch string (e.g., `ChatML <|im_end|>...<|im_start|>user`) from the tokenizer’s template by differencing, and prefill it as ordinary text through the same explicit mask/position path—no special kernel. This rebuild (tens of ms) is *off* the critical path: stopping playback plus truncation suffices for the perceived “instant stop,” and the rebuild is deferred until the next user input. The generation loop writes each new token’s KV, mask, and id back into a single accumulating cache, making assistant-side KV first-class, truncatable state across multiple turns.

C. Interrupted-History Policies

With playback truncation the history may end mid-sentence. We compare, on top of it: *naive* (leave as-is); *mark* (append an interruption marker, zero cost); *rewrite* (only for partial cuts, a 0.6B model rewrites the half sentence into a natural

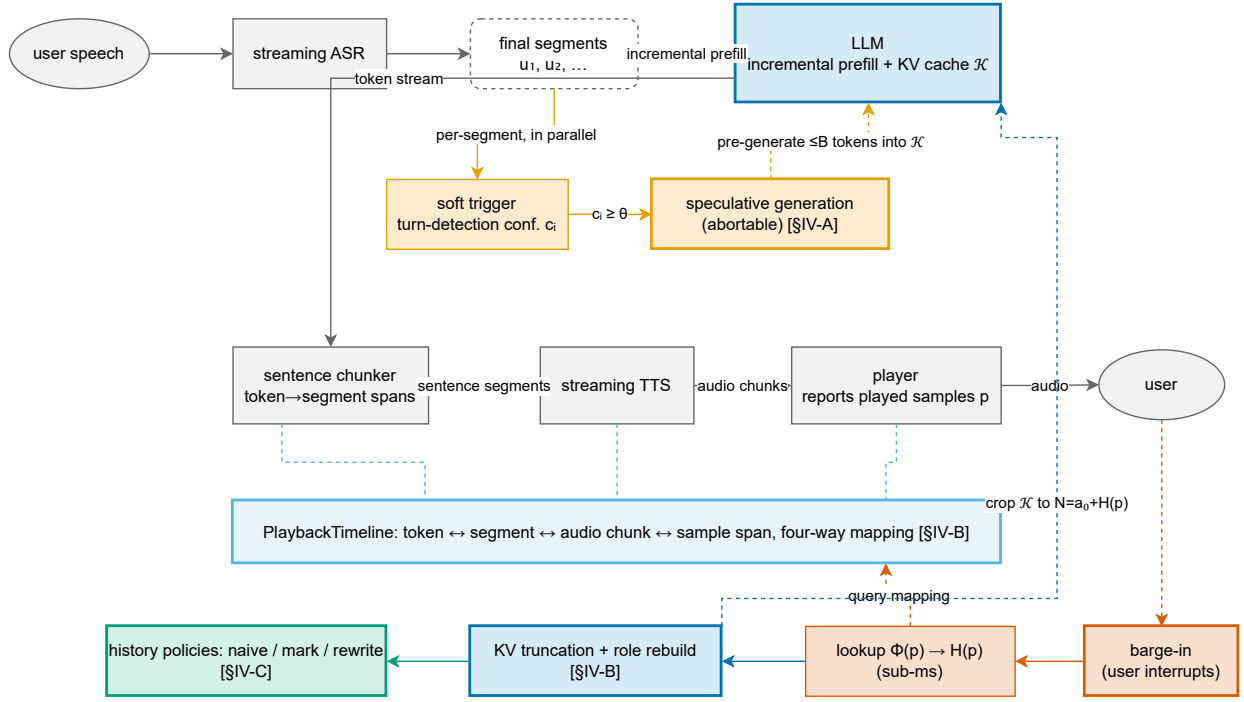


Fig. 2. System architecture. Top: input pipeline and soft-trigger-driven speculative generation (Sec. IV-A). Middle: output pipeline and the four-way mapping timeline. Bottom: barge-in chain—lookup $\Phi(p) \rightarrow H(p)$, KV truncation + role rebuild (Sec. IV-B), history policies (Sec. IV-C).

ending, constrained to add no information; the KV of the interrupted assistant span is recomputed from the rewritten text, and rewriting runs concurrently with the user’s interrupting speech).

V. EVALUATION

A. Setup

Two RTX 3090 GPUs; main LLM Qwen2-7B-Instruct [12]; soft trigger TEN Turn Detection (7.6B, text-side; calibrated pairwise AUC = 1.00 on our utterances) [11]; rewriter Qwen3-0.6B [13]; LLM judge Mistral-7B-Instruct-v0.3 [14] (a different model family from the backbone, to avoid same-family bias); streaming TTS CosyVoice2-0.5B [15] with a measured timing profile (3175 samples per non-space character at 24 kHz; first-chunk latency 2433.6ms on 3090 without TensorRT; RTF 0.513). Data derive from MultiWOZ 2.1 [16] (fixed seed): 103 multi-turn dialogues and 100 clause-split utterances, English. Barge-in is injected deterministically at playback fractions 25%/50%/75% and at clean segment boundaries (control), with the playback pointer as ground truth—fully reproducible. Conditions: System A (non-streaming baseline: full prefill after end of speech, full generation, whole-utterance synthesis), B-ours (playback truncation), B-gen (generation truncation, keeping all generated content). Synthesis-point truncation is equivalent to B-gen under our synchronous-synthesis mock and is not claimed as validated.

B. Consistency (main result)

Table I reports the unheard-content reference rate ($n=412$ per condition). Under the loose criterion B-ours is zero by

construction (the mechanism leaves no unheard segment in history)—the measured quantity is the baseline’s failure rate: 51.0% of B-gen’s follow-up replies reuse unheard surface content, and even the most conservative judge criterion leaves 2.7% of turns where the model “quotes what it never said” ($p < 10^{-3}$, Fisher exact). The rate grows the earlier the interruption (85.4% at 25% playback vs. 20.4% at 75%, rule criterion; Fig. 3). Under the strict criterion the residual tail of partial segments makes the rule detector report 51.0% for B-ours—but this is a lexical-overlap upper bound; the judge reports 2.4%, statistically indistinguishable from B-gen’s 2.9% ($p=0.83$): segment-granularity truncation carries no measurable extra cost in the specific-reference sense. Clean-boundary injections show 103/103 zero strict residue, a data-level check of the boundary semantics.

The two instruments disagree strongly on real data (Cohen’s $\kappa \approx 0.05$): MultiWOZ domain words (hotel, book, train) make the rule detector over-trigger. We therefore read the rule criterion as an upper bound and the judge as a lower bound, and arbitrate with 37 human-annotated samples: human–judge agreement is moderate-to-strong (84.6%, $\kappa = 0.649$) while human–rule agreement is no better than chance ($\kappa = -0.073$), supporting the judge as the headline instrument. Notably, humans judged 0/24 strict-criterion samples to be actual specific references, empirically vindicating segment-level truncation granularity.

C. Waste–latency frontier

Fig. 4 shows the nine-point sweep of θ ($n=100$ per point) including the never-speculate sentinel. The frontier is mono-

TABLE I
UNHEARD-CONTENT REFERENCE RATE (E3, $n=412$ PER CONDITION)

Criterion	Detector	B-ours	B-gen	p
loose	rule	0.0% (by constr.)	51.0%	1.8×10^{-79}
strict	rule	51.0%	73.3%	5.0×10^{-11}
loose	judge	0.0%	2.7%	9.1×10^{-4}
strict	judge	2.4%	2.9%	0.83 (n.s.)

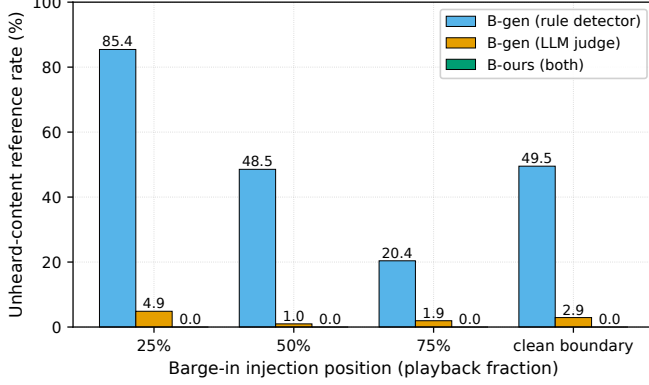


Fig. 3. Unheard-content reference rate by injection position. B-gen decreases monotonically with later interruption; B-ours is zero under both criteria—a constructive guarantee of the mechanism, not a statistical finding.

tone with a clear knee ($\theta \in [0.85, 0.97]$): at $\theta=0.92$, 4.5% waste buys TFT_{eff} 48.5 $\rightarrow$ 12.1 ms (−75%); at $\theta=0.9688$, 0.8% waste still yields −40%. At the aggressive end ($\theta \leq 0.4$) waste is 14–29% while $\text{TFT}_{\text{eff}} \approx 0$: speculation is almost always ready before the user finishes. Note the 7B “never speculate” full TFT is only 48.5 ms thanks to streaming prefill, so the *relative* gains (75–99%) generalize better than the absolute milliseconds; for larger models or longer reply prefixes the absolute gain scales up.

D. End-to-end latency

At the $\theta=0.391$ operating point ($n=100$), measured TFT : System A 27.4 ms vs. B-ours TFT_{eff} **0.6 ms** (97.9% reduction). Modeled mouth-to-ear (first-segment ready + TTS first chunk, 3090 profile): A 9080 ms (must finish generating, then synthesize the whole reply) vs. B-ours 2482 ms, of which 2434 ms is TTS first-chunk latency. This term is strongly deployment-dependent: extrapolating with the vendor-reported A100+TensorRT first-chunk figure (≈ 45 ms) brings B-ours to the ~ 100 ms range, whereas System A’s wait-for-everything cost is structural and grows with reply length.

E. KV reuse vs. re-prefill

Fig. 5 sweeps context length 275–8192 tokens (median of 5, 7B). The critical path (lookup + crop) is 0.31–0.34 ms, constant in context length; role rebuild (25–47 ms) is off the critical path; re-prefilling the heard-boundary context grows linearly, reaching 1.86 s at 8k—a 39.7 $\times$ gap that widens as multi-turn context grows.

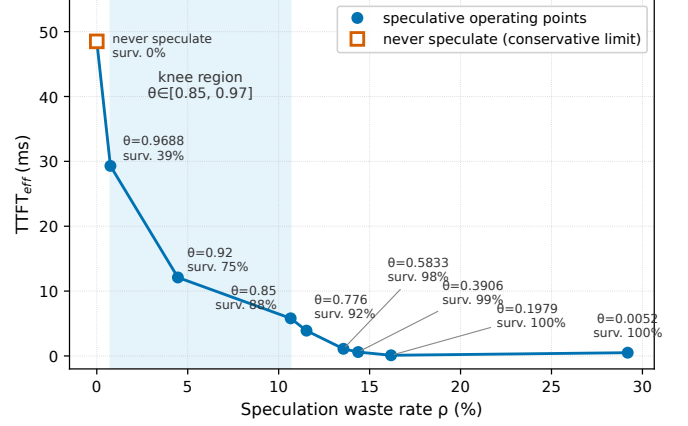


Fig. 4. Speculation waste rate vs. TFT_{eff} : a continuously tunable frontier; the open square is the never-speculate conservative limit.

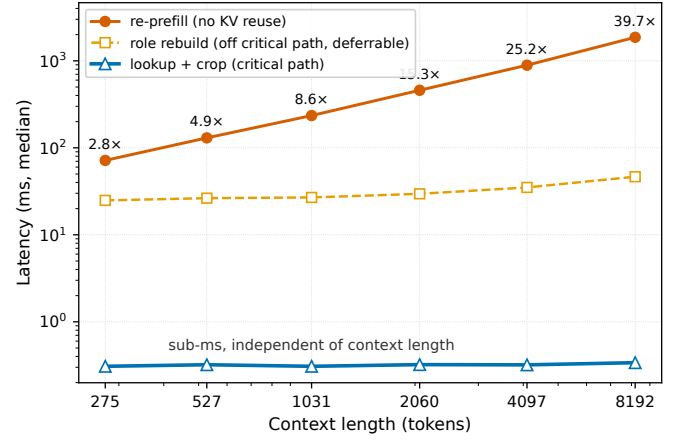


Fig. 5. Barge-in response paths vs. re-prefill across context lengths (log-log). Lookup+crop is sub-ms and length-independent; re-prefill grows nearly linearly (39.7 $\times$ at 8k).

F. Interrupted-history policies (honest null)

Judge coherence (1–5, $n=100$ per policy): naive 3.76, rewrite 3.62, mark 3.29; rewriting is *not* better than naive truncation (Wilcoxon $p=0.32$). Plausible reasons: the 7B backbone is already robust to half-sentence history; the 0.6B rewriter’s paraphrasing dilutes contextual cues. The engineering claims still hold—rewrite latency mean 639 ms, P90 937 ms, hideable inside the user’s interrupting speech—so we position rewriting as an exploratory mechanism pending stronger rewriters and pairwise-preference evaluation.

VI. DISCUSSION AND LIMITATIONS

Relation to commercial systems. The differences are operation level (explicit KV truncation preserving cross-turn KV reuse vs. transcript deletion), position source (player-reported sample counts and reverse mapping vs. real-time-rate assumptions or client-reported offsets), and inspectability (all mappings and decisions logged; experiments reproducible). None of these support a claim that commercial systems lack

the capability. *Instrumentation*. The rule detector over-triggers on domain words; our upper-bound/lower-bound/arbitration protocol mitigates but a larger human study would tighten the interval. *Timing model*. Mouth-to-ear is modeled from a measured profile; first-chunk latency varies by two orders of magnitude across inference stacks, so cross-system comparisons need care. *Scope*. Experiments cover English task-oriented dialogues with deterministic injected barge-in; open-domain speech, natural interruption behavior, and Chinese-language paths are future work. The mechanisms themselves bind to no specific stack: any sentence-level streaming TTS, any causal LM in the `DynamicCache` ecosystem, and any confidence-emitting endpoint model can be substituted.

VII. CONCLUSION

We formalized the generation–synthesis–playback mismatch that makes cascaded voice agents misremember what they said, and presented an open, reproducible cascade in which dialogue history provably equals what the user heard: speculative scheduling makes endpoint detection a tunable waste–latency knob (48.5→12.1 ms at 4.5% waste); playback-aware KV truncation answers barge-in in sub-millisecond time independent of context length (39.7× over re-prefill at 8k); and a dual-criterion protocol with human arbitration shows naive truncation resurfaces unheard content in 51%/2.7% (surface/specific) of interrupted turns while our mechanism eliminates it by construction, with no measurable cost from segment-granularity truncation. Code, data derivations, and per-turn instrumentation are open-sourced for reproduction.

ACKNOWLEDGMENT

Large language models were used to assist with literature retrieval verification, figure scripting, and language polishing of this manuscript; all technical content, experiments, and conclusions were produced and verified by the authors.

REFERENCES

- [1] OpenAI, “Realtime API — conversations,” <https://developers.openai.com/api/docs/guides/realtime-conversations>, 2025, accessed: 2026-07-21.
- [2] Microsoft Azure, “Voice Live auto-truncation,” <https://learn.microsoft.com/en-us/azure/ai-services/speech-service/how-to-voice-live-auto-truncation>, 2025, accessed: 2026-07-21.
- [3] LiveKit, “Agents — events and interruption handling,” <https://docs.livekit.io/agents/build/events/>, 2025, accessed: 2026-07-21.
- [4] W. Zou, Y. Miao, Z. Ma, J. Xu *et al.*, “LTS-VoiceAgent: A listen-think-speak framework for efficient streaming voice interaction via semantic triggering and incremental reasoning,” 2026, arXiv:2601.19952.
- [5] L. Mai, “RelayS2S: A dual-path speculative generation for real-time dialogue,” 2026, arXiv:2603.23346.
- [6] A. Défossez, L. Mazaré, M. Orsini, A. Royer, P. Pérez, H. Jégou, E. Grave, and N. Zeghidour, “Moshi: A speech-text foundation model for real-time dialogue,” 2024, arXiv:2410.00037.
- [7] FireRed Team, “FireRedChat: Streaming personalized VAD for barge-in,” 2025, arXiv:2509.06502.
- [8] Hugging Face, “Transformers documentation: KV cache strategies,” https://huggingface.co/docs/transformers/en/kv_cache, 2025, accessed: 2026-07-21.
- [9] J. Li, J. Lou, and J. Li, “IntentKV: Cross-turn intent-aware KV cache pruning for agent inference,” 2026, arXiv:2606.09916.
- [10] C. Hooper, M. Kang, S. Moon *et al.*, “Speculative interaction agents: Building real-time agents with asynchronous I/O and speculative tool calling,” 2026, arXiv:2605.13360.

- [11] TEN Framework, “TEN turn detection: Turn detection for full-duplex dialogue communication,” <https://github.com/TEN-framework/ten-turn-detection>, 2025, accessed: 2026-07-21.
- [12] A. Yang *et al.*, “Qwen2 technical report,” 2024, arXiv:2407.10671.
- [13] —, “Qwen3 technical report,” 2025, arXiv:2505.09388.
- [14] A. Q. Jiang, A. Sablayrolles, A. Mensch *et al.*, “Mistral 7B,” 2023, arXiv:2310.06825.
- [15] Z. Du *et al.*, “CosyVoice 2: Scalable streaming speech synthesis with large language models,” 2024, arXiv:2412.10117.
- [16] M. Eric, R. Goel, S. Paul, A. Sethi, S. Agarwal, S. Gao, A. Kumar, A. Goyal, P. Ku, and D. Hakkani-Tür, “MultiWOZ 2.1: A consolidated multi-domain dialogue dataset with state corrections and state tracking baselines,” in *Proc. 12th Language Resources and Evaluation Conf. (LREC)*, 2020, pp. 422–428.